# Salesforce Certified Tableau Data Analyst - Answer Key & Explanations

**Practice exam:** Analytics-DA-201 | **Questions:** 60 | **Pass mark:** 65% (39 / 60)

### How to score yourself

| Rule | Detail |
|---|---|
| Passing threshold | **65% -> 39 of 60 correct** |
| Multi-select items (SELECT 2 / SELECT 3) | **No partial credit** - the item is correct only if *every* selected option is right and none is missing |
| Suggested workflow | Take the paper under timed conditions (105 min), mark it with this key, then study every missed question plus its **Key point** line |

### Answer grid (quick reference)

| Q | Ans | Q | Ans | Q | Ans | Q | Ans | Q | Ans |
|---|---|---|---|---|---|---|---|---|---|
| 1 | B | 13 | B | 25 | B | 37 | A | 49 | A |
| 2 | B | 14 | B | 26 | A | 38 | A | 50 | A |
| 3 | A, C | 15 | B | 27 | A | 39 | A | 51 | A |
| 4 | B | 16 | B | 28 | A | 40 | A | 52 | A |
| 5 | B | 17 | A | 29 | A | 41 | C | 53 | A |
| 6 | B | 18 | A | 30 | A | 42 | A | 54 | A |
| 7 | B | 19 | A | 31 | A | 43 | A | 55 | A |
| 8 | A | 20 | B | 32 | B | 44 | B | 56 | B |
| 9 | C | 21 | B | 33 | C | 45 | B | 57 | B |
| 10 | A | 22 | A | 34 | A | 46 | A | 58 | B |
| 11 | B | 23 | A, C | 35 | A | 47 | A | 59 | C |
| 12 | D | 24 | B | 36 | A | 48 | A, B | 60 | B |

---

## Section 1 - Connect to and Transform Data

**Q1. Answer: B** - A **live connection** queries the database directly, so every view render reflects the database's current state. A and C are extracts (snapshots): even a 15-minute or incremental refresh has a lag. D is about combining sources, not freshness.

> **Key point:** "Real time / always current / updates every minute" -> **live connection**. Only live guarantees real-time values. Extracts are fast but stale between refreshes.

**Q2. Answer: B** - A slow source with data that changes only once a day is the textbook case for an **extract with a scheduled refresh** just after the 2:00 AM load. Users then query the fast local Hyper engine. A keeps the slow round-trips. C (custom SQL) still runs against the slow source. D (blending) addresses combining sources, not speed.

> **Key point:** "Slow source + periodic updates + performance" -> **extract + scheduled refresh**.

**Q3. Answer: A and C** - Extracts store data in Tableau's columnar **Hyper** engine (faster) and can be used **offline/portable**. B is false - extracts are snapshots, not real time. D is false - extracts must be refreshed to stay current.

> **Key point:** Extract = **fast + portable but stale**. Live = **current but dependent on source speed**.

**Q4. Answer: B** - **Data -> Replace Data Source...** re-points a workbook to a new source; when field names match, existing charts keep working. A is manual and error-prone, C would append rows (duplicating data), D discards all work.

> **Key point:** Re-pointing a whole workbook to a same-schema source = **Replace Data Source**.

**Q5. Answer: B** - A **relationship** keeps tables at their native granularity, prevents customer-row duplication, and lets Tableau decide the join per query context. A (inner join) risks duplication and drops unmatched rows. C (blend) is a workaround. D (union) requires identical structure.

> **Key point:** Default modern choice = **relationships**. Joins are for deliberate row-level merges; unions require identical columns.

**Q6. Answer: B** - A **left join** with Orders on the left keeps every order, filling unmatched return fields with NULL. A (inner) drops orders with no return. C (right) keeps all returns instead. D (full outer) also keeps unmatched rows from both sides (more than needed here).

> **Key point:** "Keep every row from table X even when no match" -> **left join with X on the left**.

**Q7. Answer: B** - **Pivoting the twelve month columns into rows** produces a tidy long table with a Month dimension and one Revenue measure, ideal for time trends. Splitting splits strings; unioning columns is not how unions work; the Data Interpreter cleans formatting.

> **Key point:** Many columns storing the *same kind* of value -> **pivot columns to rows**.

**Q8. Answer: A** - **Union** stacks tables that share the same structure. Joins merge columns; relationships relate different tables; blends combine aggregated sources.

> **Key point:** "Same columns, append rows" -> **union**. "Match rows and add columns" -> **join**.

**Q9. Answer: C** - An **extract filter** defines which rows are written into the extract at creation/refresh. A context filter applies within a worksheet, a table-calculation filter hides marks after computation, and a quick filter is a view control.

> **Key point:** Rows physically stored in the extract = **extract filter** (a data source filter can also reduce rows pulled, but the extract filter is the one tied to extract creation).

**Q10. Answer: A** - A **data source filter** is defined on the data source, applies to all sheets/users, and is pushed to the underlying query. B and D are per-worksheet; C is a per-sheet limit.

> **Key point:** One filter for the whole source = **data source filter**.

**Q11. Answer: B** - A `.csv` file is universally readable without Tableau. `.hyper` requires Tableau, `.twbx` is a packaged workbook, and `.tfl` is just the flow definition.

> **Key point:** Output for a non-Tableau consumer -> `.csv` (or a database table).

**Q12. Answer: D** - **Common Characters** groups values that share the same characters regardless of order or separators - exactly these four variants. **Spelling** matches near-identical strings; **Pronunciation** matches values that *sound* alike; **Manual Selection** is hand-picked.

> **Key point:** Same characters, different order/separators -> **Common Characters**. Sound-alike -> **Pronunciation**. Similar spelling -> **Spelling**.

**Q13. Answer: B** - The **Data Interpreter** detects and removes titles, merged-header noise, sub-totals, and footnotes from human-formatted Excel (and some CSV/PDF) files.

> **Key point:** Messy human-formatted spreadsheet -> **Data Interpreter**.

**Q14. Answer: B** - `Customer ID` is being treated as a continuous measure and summed. **Converting it from a measure to a dimension** lists each ID as a discrete category. Changing aggregation to COUNTD would still aggregate (one number), an LOD is unnecessary, and number formatting does not change the field role.

> **Key point:** "List individual values, not a total" = the field must be a **dimension**.

---

# Section 2 - Explore and Analyze Data

## LOD expressions

**Q15. Answer: B** - `{FIXED : SUM([Sales])}` (a table-scoped FIXED with no dimensions) computes the grand total **before** dimension filters run, so filtering a sub-category out does not change the denominator. A uses `TOTAL()`, a **table calculation** that is computed *after* dimension filters, so the denominator would shrink. C with `INCLUDE [Sub-Category]` returns each sub-category's own total (the ratio becomes 1). D (`EXCLUDE`) is applied *after* dimension filters, so it too would change.

> **Key point:** Denominator that ignores dimension filters -> **FIXED** (LODs run before dimension filters). `TOTAL()` is a table calc and runs last.

**Q16. Answer: B** - `{INCLUDE [Customer Name] : SUM([Sales])}` computes sales per customer, then aggregating as **AVG** rolls it up to the region, giving average sales per customer. A (`FIXED [Region] : AVG`) does not break the data out by customer first. C removes the region. D is a plain view-level average.

> **Key point:** "Average per , shown at a coarser dimension" -> **INCLUDE** the finer dimension, then aggregate (AVG).

**Q17. Answer: A** - `{EXCLUDE [Region] : SUM([Sales])}` removes the region from the view level of detail, giving total sales per month across all regions. B excludes the wrong dimension. C fixes *by* region (opposite). D adds the month that is already present.

> **Key point:** EXCLUDE = "**subtract this dimension** from the view's granularity."

**Q18. Answer: A** - The **nested LOD** first computes `SUM(Sales)` per month with the inner `{FIXED [month] : ...}`, then the outer `{EXCLUDE [month] : AVG(...)}` averages those monthly values across the whole period. B mixes an LOD with a view-level average incorrectly. C has no per-month structure. D is a single constant.

> **Key point:** Nested LOD = an LOD **inside** another LOD ( `{EXCLUDE ... : AVG({FIXED ... : SUM(...)})}` ). Compute the granular value first, then re-aggregate.

**Q19. Answer: A** - **FIXED LODs are evaluated before dimension filters** in the order of operations, so an ordinary dimension filter does not reach them. To make a filter affect a FIXED LOD, add the filter to **context** - or rewrite the LOD as INCLUDE/EXCLUDE.

> **Key point:** Order of operations: extract -> data source -> context -> **FIXED LOD** -> dimension filters -> INCLUDE/EXCLUDE LOD -> measure filters -> table calculations.

**Q20. Answer: B** - **FIXED** uses only the dimensions you declare and ignores the view; **INCLUDE** uses the declared dimensions *plus* whatever is in the view. A reverses them. C is false. D is false - INCLUDE/EXCLUDE are always measures, while FIXED can be a dimension or a measure.

> **Key point:** FIXED = "regardless of the view." INCLUDE = "add a dimension to the view." EXCLUDE = "remove a dimension from the view."

**Q21. Answer: B** - **INCLUDE** adds a dimension that is not already in the view. FIXED fixes an independent level; EXCLUDE removes a dimension; TOTAL is a table calculation, not an LOD keyword.

> **Key point:** INCLUDE brings an *off-screen* dimension into the calculation.

**Q22. Answer: A** - `{FIXED [Region] : COUNTD([Customer ID])}` computes distinct customers at the region level and repeats that value on every sub-category row. B (INCLUDE) would add region *to* the view's dimensions and produce per-sub-category counts. C removes region. D is a view-level count.

> **Key point:** FIXED gives a **constant per dimension member** across all finer rows in the view.

**Q23. Answer: A and C** - FIXED LODs are affected by **context, data source, and extract filters** (A). INCLUDE and EXCLUDE LODs are evaluated *after* dimension filters, so they **are** affected by them (C). B is false (FIXED ignores ordinary dimension filters). D is false (LODs are not table calculations).

> **Key point:** FIXED ignores dimension filters; INCLUDE/EXCLUDE respect them. Context/data-source/extract filters affect all.

**Q24. Answer: B** - `MIN([Order Date])` resolves to a date **dimension**, so the FIXED LOD is added to the Dimensions area. FIXED LODs can be either; INCLUDE and EXCLUDE are always measures.

> **Key point:** FIXED result type follows the underlying aggregate - date -> dimension, numeric measure -> measure.

## Calculated fields

**Q25. Answer: B** - `DATEPARSE("yyyy-MM-dd", [Order Date Text])` converts a formatted string into a date/datetime. `DATE()` will not parse an arbitrary text format, `MAKEDATE` needs numbers, and `DATEADD` requires an existing date.

> **Key point:** Text with a known pattern -> **DATEPARSE(format, string)**.

**Q26. Answer: A** - `DATEDIFF('day', [Order Date], [Ship Date])` returns the number of day boundaries between two dates. B subtracts strings (invalid), C truncates a date, and D adds dates.

> **Key point:** Difference between two dates -> **DATEDIFF(date_part, start, end)**.

**Q27. Answer: A** - `SPLIT([Customer Name], ",", 2)` returns the second token after the comma (`" John"`), and `TRIM` removes the leading space. B/C/D use fixed character positions and fail for variable-length names.

> **Key point:** Extract a piece of a delimited string -> **SPLIT(field, delimiter, token_number)**, then `TRIM` if needed.

**Q28. Answer: A** - `REPLACE([Company], "Inc.", "Incorporated")` substitutes every occurrence. FIND returns a position, CONTAINS returns a Boolean, and SPLIT returns a token.

> **Key point:** Find-and-replace text = **REPLACE(string, substring, replacement)**.

**Q29. Answer: A** - `IF ... ELSEIF ... ELSE ... END` handles the three ordered conditions correctly. B collapses "0" into "Loss." C uses `CASE` with a numeric field against string labels. D reverses the logic and loses "Break-even."

> **Key point:** Multiple ordered ranges -> **IF / ELSEIF / ELSE**. `CASE` is cleaner for exact-value matches.

**Q30. Answer: A** - `AND` requires both conditions to be true. B uses `OR` (either condition passes). C inverts the result. D ignores Sales entirely.

> **Key point:** Both conditions must hold -> **AND**. Either -> **OR**.

**Q31. Answer: A** - `ROUND(AVG([Discount]), 1)` computes the average, then rounds to **one** decimal place. B rounds to a whole number, C rounds each row before averaging (different result), and D truncates to an integer.

> **Key point:** `ROUND(number, decimals)` - the second argument is the number of decimal places.

**Q32. Answer: B** - `INT([Store Number])` converts the text to a whole number. `STR` does the opposite, `DATE` is for dates, and `SPLIT` is for strings.

> **Key point:** Type conversion: **INT** (integer), **FLOAT** (decimal), **STR** (text), **DATE** (date), **DATETIME** (date-time).

**Q33. Answer: C** - `COUNTD` counts **distinct** values, so repeated customer IDs are counted once. `COUNT` counts every row (over-counting), `SUM` adds the IDs (meaningless), and `ATTR` tests for a single value.

> **Key point:** Unique values -> **COUNTD**.

## Table calculations

**Q34. Answer: A** - `WINDOW_AVG(SUM([Sales]), -2, 0)` averages the current row and the two preceding rows - a 3-period moving average. B sums instead of averages, C is not a true moving average, and D ignores the window.

> **Key point:** 3-period moving average = **`WINDOW_AVG(expr, -2, 0)`**.

**Q35. Answer: A** - `RUNNING_SUM(SUM([Sales]))` accumulates down the view; addressing it along **Month** and partitioning by **Year** makes it reset each year (year-to-date). B is a grand total, C is a 12-month average, and D is a one-period look-back.

> **Key point:** YTD = **RUNNING_SUM** addressed along the time axis and **partitioned by Year**.

**Q36. Answer: A** - `LOOKUP(expr, -1)` returns the previous month's value; subtracting and dividing by it gives percent difference. B gives the absolute difference, C gives percent of total, and D gives a rank.

> **Key point:** Percent difference = **`(current - LOOKUP(expr, -1)) / LOOKUP(expr, -1)`**.

**Q37. Answer: A** - `INDEX()` returns the current row's position within the partition, starting at 1. `RANK` ranks by a value, while `FIRST()` and `LAST()` return offsets from the first/last row (0 at that row).

> **Key point:** Position counter -> **INDEX()**. Value rank -> **RANK()**.

**Q38. Answer: A** - Compound growth per period is `(Last / First)^(1/(n-1)) - 1`. B is simple (non-compounded) growth, C is an average, and D compounds the wrong quantity.

> **Key point:** CAGR = `POWER(Last/First, 1/(n-1)) - 1`.

### Filters, parameters, sets, and bins

**Q39. Answer: A** - Both are dimension filters and run at the same stage, so the Top 10 is computed globally. Adding the **City** filter to **context** forces it to run first, scoping the Top 10 to the chosen city. Changing Top to Bottom does not help, table-calculation filters run later, and a parameter does not fix the order.

> **Key point:** To make one dimension filter run before another -> **Add to Context**.

**Q40. Answer: A** - Create a **parameter**, then use it as the value of the **reference line**. B cannot take user input, C filters data, and a set action is for selection interactions.

> **Key point:** User-entered value driving a reference line -> **parameter + reference line**.

**Q41. Answer: C** - A histogram needs a **bin** built from a continuous measure: the bin goes on Columns and a count goes on Rows. Sets, groups, and hierarchies do not create the equal-width buckets a histogram needs.

> **Key point:** Histogram = **bin (equal-width buckets) + count**.

---

## Section 3 - Create Content

**Q42. Answer: A** - Placing both measures on Rows and selecting **Dual Axis** creates a combined view with **two independent axes**, letting you synchronize them and assign bars vs. lines via the Marks card. B shares one scale, C filters data, and D produces separate views.

> **Key point:** Two independent axes in one view = **dual axis** (drag the second measure to the right of the first, then Dual Axis).

**Q43. Answer: A** - A scatter plot needs a measure on each axis. `Customer Name` on **Detail** creates one mark per customer, and `Profit Ratio` on **Color** encodes it. B/C/D omit one of the two axes or place fields incorrectly.

> **Key point:** Scatter plot = **measure on Columns + measure on Rows + dimension on Detail**.

**Q44. Answer: B** - Bins from a continuous measure set the x-axis buckets, and a **count** on Rows gives the frequency per bucket. A produces a normal bar chart, C a dual-axis chart, and D a treemap.

> **Key point:** Histogram = **bin on Columns + COUNT on Rows**.

**Q45. Answer: B** - A **treemap** shows nested rectangles whose **area** encodes a measure such as sales. A highlight table uses color in a grid, a Gantt chart shows durations, and a box plot shows distributions.

> **Key point:** Nested rectangles sized by a measure = **treemap**.

**Q46. Answer: A** - A **Gantt** chart puts a dimension (Task) on Rows and a continuous date on Columns, with a duration measure on **Size** to draw each bar. B/C/D are different chart families.

> **Key point:** Gantt = **dimension on Rows + continuous date on Columns + duration on Size**.

**Q47. Answer: A** - A **manual (custom) sort order** lets you arrange dimension members in a business-specific sequence by dragging them in the Sort dialog. B computes values, C filters, and D is for interactions.

> **Key point:** Non-alphabetical, non-measure order -> **manual/custom sort**.

**Q48. Answer: A and B** - A **fixed-size container** holds its dimensions when the dashboard resizes (A), and containers can hold sheets, other containers, text, images, and more (B). C is false (containers hold many items), and D is false (tiled and floating items can be mixed).

> **Key point:** Containers are the structural building blocks of a dashboard and can nest.

**Q49. Answer: A** - A **filter action** makes a selection in one sheet filter target sheets. A highlight action only emphasizes marks, a URL action opens a link, and a tooltip only shows text.

> **Key point:** Selection that **filters other sheets** -> **Filter action**.

**Q50. Answer: A** - A **URL action** opens a web page; you insert the store field into the URL so the selected value is passed through. Filter/highlight actions do not open URLs.

> **Key point:** Selection opens a web page -> **URL action** (with the field inserted into the URL).

**Q51. Answer: A** - **Parameter actions** and **set actions** let a viewer's selection write a value into a parameter or set, which can then drive highlighting or filtering elsewhere. Extract filters and the Data Interpreter are data-prep features; dual-axis sync is unrelated.

> **Key point:** Selection writes a value -> **parameter action / set action**.

**Q52. Answer: A** - **Dynamic zone visibility** shows or hides a zone automatically based on a Boolean calculation or parameter - no button required. Show/hide buttons require a viewer click, navigation buttons jump between locations, and subscriptions email content.

> **Key point:** Automatic, calculation-driven show/hide -> **dynamic zone visibility**. Viewer-driven -> **show/hide button**.

**Q53. Answer: A** - A **show/hide button** lets the viewer toggle an object. Dynamic zone visibility is designer-driven; URL actions open links; reference lines are analytics objects.

> **Key point:** Viewer toggles an object -> **show/hide button**.

**Q54. Answer: A** - **Viz in Tooltip** embeds a small worksheet inside a tooltip that appears on hover. Reference bands, box plots, and forecasts are analytical overlays, not hover charts.

> **Key point:** Chart shown on hover = **Viz in Tooltip**.

**Q55. Answer: A** - Custom color palettes are defined in `Preferences.tps` in the **My Tableau Repository** folder (Documents -> My Tableau Repository), then Tableau is restarted. Editing workbook XML is not the supported method.

> **Key point:** Custom palette -> **Preferences.tps** in My Tableau Repository.

## Section 4 - Publish and Manage Content

**Q56. Answer: B** - Publishing a **data source** creates a single governed, reusable dataset that many workbooks can connect to. Embedding a copy in each workbook duplicates definitions; a story and a screenshot are not data sources.

> **Key point:** One shared governed dataset -> **published data source**.

**Q57. Answer: B** - Extract refresh schedules require the data source to contain an **extract**; a purely live connection has nothing to refresh. There is no 1 GB limit, and Creators/admins (per permissions) can schedule refreshes.

> **Key point:** Scheduled **extract** refresh -> the source must have an **extract**.

**Q58. Answer: B** - **Alerts** fire when a metric crosses a threshold the user sets; **subscriptions** email a scheduled snapshot (image/PDF) of a view or workbook. A reverses them. C and D are false - both are available to users with appropriate permissions and do not require live connections.

> **Key point:** Alert = **threshold event**. Subscription = **scheduled snapshot email**.

**Q59. Answer: C** - The **Creator** site role can connect to data, create/publish content, and **publish flows**. Viewers and Explorers have narrower capabilities and cannot publish flows.

> **Key point:** Connect + create + **publish flows** -> **Creator**.

**Q60. Answer: B** - A **Custom View** saves a viewer's own filter/sort/selection state on a published view without affecting anyone else. Duplicating the workbook requires publish rights, editing the data source filter changes it for everyone, and reverting would discard changes.

> **Key point:** Personal saved view state -> **Custom View**.

---

## Score Interpretation

| Score | Meaning |
|---|---|
| 52-60 | Excellent - exam-ready across all domains |
| 45-51 | Strong - review the domains you missed |
| 39-44 | Passing, but close - drill Domain 2 (LODs / table calcs) hard |
| Below 39 | Not yet - reread the Tableau Help pages for Order of Operations and LOD expressions, then retake this paper |

**Highest-yield topics to review if you scored below 39**

1. **Tableau Order of Operations:** extract -> data source -> context -> **FIXED LOD** -> dimension filters -> **INCLUDE/EXCLUDE LOD** -> measure filters -> table calculations. Most LOD/filter questions hinge on this.
2. **FIXED vs INCLUDE vs EXCLUDE** - when each is used, and what they do to the view's granularity.
3. **Nested LODs** - compute a granular value, then re-aggregate.
4. **Table calc building blocks:** `WINDOW_AVG`, `RUNNING_SUM`, `LOOKUP`, `INDEX`, `RANK`, `TOTAL`, `FIRST`/`LAST`, and addressing vs. partitioning.

5. **Calculated-field functions:** `DATEPARSE`, `DATEDIFF`, `SPLIT`, `REPLACE`, `ROUND`, `INT/FLOAT/STR`, `COUNTD`, and `IF/ELSEIF` vs `CASE`.

*End of answer key.*